

Python Modules & Packages

30 High-Level Interview & Technical Screening Questions

QUESTIONNAIRE

PART 1: CORE FUNDAMENTALS & IMPORT SYNTAXES (Q1 – Q7)

- 01 What is a module in Python, and why is it used?
- 02 What is the difference between a module and a package in Python?
- 03 Why do we need modules when Python already supports functions and classes?
- 04 What happens internally when Python imports a module?
- 05 What is the difference between `import module` and `from module import function` ?
- 06 What is the difference between `import module` and `from module import *` ? Why is the latter generally discouraged?
- 07 What is the purpose of the `as` keyword while importing a module?

PART 2: MODULE TYPES, ECOSYSTEM & PACKAGE ANATOMY (Q8 – Q15)

- 08 What is a built-in module in Python? Give examples.
- 09 What is a user-defined module? How is it different from a built-in module?
- 10 What is a third-party module in Python? How do we generally install one?
- 11 What is the role of `pip` in Python modules and packages?
- 12 What is the difference between a Python package and a third-party library?
- 13 What is `__init__.py` , and what role does it play inside a package?
- 14 Is `__init__.py` mandatory for every Python package? Explain.
- 15 What is a subpackage in Python?

PART 3: SEARCH PATHS, RELATIVE IMPORTS & RESOLUTION (Q16 – Q21)

- 16 What is the difference between absolute import and relative import?
- 17 What do `.`, `..`, and `...` represent in relative imports?
- 18 What is the Python module search path? How does Python find an imported module?
- 19 What is `sys.path`, and why is it important for imports?
- 20 What is `PYTHONPATH`? How is it related to module searching?
- 21 What happens if two modules have the same name in different locations?

PART 4: CIRCULAR IMPORTS, RUNTIME CACHING & ARCHITECTURE (Q22 – Q30)

- 22 What is a circular import? Why does it occur, and what problems can it cause?
- 23 How can circular imports be avoided or resolved?
- 24 What is module caching in Python? Why doesn't Python normally execute an imported module again every time it is imported?
- 25 What is `sys.modules`? What information does it contain?
- 26 What is the difference between importing a module and executing a Python file directly?
- 27 What is the purpose of `if __name__ == "__main__":` in a module?
- 28 What is namespace pollution in Python imports? How can it be avoided?
- 29 What is the difference between a package, module, library, and framework?
- 30 Suppose a large Python project contains hundreds of modules. How would you organize its packages and imports to keep the project maintainable?